

Project Progress Report (BTP)

Adaptive Reverse Proxy & Load Balancer

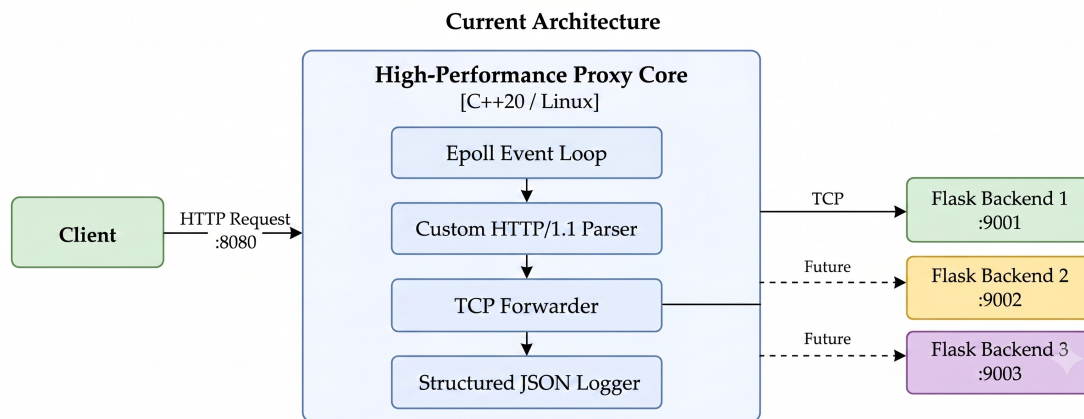
Aditya Prakash (Roll No. 2302CS11)

Working Repository: [01prakash-aditya/adaptive_lb_rp](https://github.com/01prakash-aditya/adaptive_lb_rp)

1. Current Project Status

The initial development phase of the project has been successfully completed. The primary objective of this phase was to establish a functional, high-performance reverse proxy capable of accepting client HTTP requests, forwarding them to a backend server, and returning the backend response reliably.

The proxy currently operates with a **single backend** and provides the core infrastructure required for the subsequent multi-backend load-balancing phase.



2. Work Completed

2.1 Event-Driven Proxy Core

A high-performance TCP server has been implemented in **C++20** using Linux `epoll`. The server uses non-blocking sockets and an event-driven architecture, allowing it to handle multiple client connections without relying on a thread-per-connection model.

2.2 HTTP/1.1 Request Processing

A custom incremental HTTP/1.1 parser has been implemented to process incoming requests safely and incrementally. The parser currently supports:

- HTTP request line and header parsing.
- Content-Length based request bodies.
- Transfer-Encoding: `chunked`.
- Incremental processing of data received over multiple TCP reads.

2.3 Backend Request Forwarding

The proxy successfully establishes communication with a backend server, forwards the client's HTTP request, receives the backend response, and returns the response to the client.

Failure scenarios are also handled through appropriate HTTP responses, including timeout and unavailable-backend cases such as 502 Bad Gateway and 504 Gateway Timeout.

2.4 Observability and Request Tracing

Structured JSON logging has been implemented to provide visibility into request processing. The logs capture important information including:

- HTTP method and request information.
- Response status code.
- Request processing latency.
- Unique request identifiers.

The proxy also injects X-Request-ID and X-Forwarded-For headers to support request tracing between the client and backend.

2.5 Containerized Development Environment

The complete development environment has been containerized using **Docker and Docker Compose**. The environment contains:

- The C++ reverse proxy.
- Three Python Flask dummy backend services.
- A shared Docker network for service-to-service communication.

This provides a reproducible environment for testing the proxy and will also serve as the foundation for the multi-backend load-balancing experiments.

3. Testing and Verification

The implemented functionality has been verified using:

- **10 Unit Tests** covering core proxy and parsing functionality.
- **5 Integration Tests** covering communication between the proxy and backend services.
- Manual end-to-end testing using standard HTTP clients such as curl.

The following commands can be used to reproduce the basic verification.

3.1 Start the Environment

```
$ docker-compose -f docker/docker-compose.yml up --build
```

This starts the reverse proxy and the backend services on a shared Docker network.

3.2 Verify HTTP Request Forwarding

```
# GET request
$ curl http://localhost:8080/

# POST request with JSON body
$ curl -X POST -d '{"key":"value"}' \
-H "Content-Type: application/json" \
http://localhost:8080/echo

# Health check
$ curl http://localhost:8080/health
```

These tests verify basic request forwarding, POST body handling, and backend health communication.

3.3 Verify Request Logging

```
$ docker logs reverse_proxy
```

The generated logs can be inspected to verify request IDs, HTTP methods, response status codes, and request latency.

4. Planned Work – Multi-Backend Load Balancing

With the single-backend request pipeline implemented and verified, the next phase will extend the proxy to support multiple backend servers.

The following load-balancing strategies are planned for implementation and evaluation:

1. **Round Robin** – distribute requests sequentially across available backends.
2. **Weighted Round Robin** – distribute requests according to configurable backend weights.
3. **Least Connections** – route new requests to the backend currently handling the fewest active connections.
4. **IP Hash** – consistently map clients to backend servers using a hash of the client IP address.

The implementations will subsequently be evaluated using measurable performance metrics such as request distribution, latency, throughput, connection utilization, and behavior under backend failures.